

L'épopée des silences

Spécification technique - Markov Engine V1

Métadonnée	Valeur
Projet	L'épopée des silences
Ancien nom	RPG_ESI07
Document	Spécification technique détaillée - Markov Engine
Version document	1.0
Version cible	alpha-0.0.5
Service propriétaire	Game Engine Service
Module cible	Leds.GameEngine.Domain.Markov
Statut	Spécification préalable à implémentation
Date	2026-06-03

Décision structurante

Le Markov Engine est un composant fondateur du gameplay serveur-autoritaire. Il ne doit pas être implémenté comme un helper de tirage pondéré. Il doit manipuler une vraie matrice de transition, versionnée, validée, testable et déterministe.

Sommaire

1. Objet et justification
2. Fondement mathématique obligatoire
3. Positionnement architectural
4. Modèle conceptuel final cible
5. Etats narratifs et garde-fous non médicaux
6. Déterminisme et reproductibilité
7. Validation des matrices
8. Opérations Markov obligatoires
9. Diagrammes de fonctionnement
10. API domaine cible
11. Traces, Event Sourcing et auditabilité
12. Relation avec Catalog et Lois du Palais
13. Stratégie de tests
14. Roadmap d'intégration
15. Critères d'acceptation

1. Objet et justification

La matrice de Markov doit devenir une fondation technique et ludique de la v2. Les runs du jeu sont reproductibles par seed, guidées par une version de générateur et une version de matrice. Le Markov Engine doit donc être posé avant de construire les modules qui s'appuieront sur lui : génération du Palais, sélection d'événements, attitudes PNJ, narration et adaptation d'Him'Lit.

Objectifs principaux

- fournir une vraie matrice de transition P, pas un simple sélecteur aléatoire;
- garantir le déterminisme serveur à partir de seed, scope, matrixKey, matrixVersion, état courant et step;
- supporter à la fois la transition discrète observable et l'évolution complète d'une distribution;
- rester générique et indépendant des concepts Room, Node, PNJ, Combat ou Catalog;
- permettre une implémentation proche de la version finale, avec versioning et traçabilité dès le départ.

Non-objectifs du MVP

- pas de génération réelle de Room dans cette PR;
- pas de système PNJ complet dans cette PR;
- pas de Hidden Markov Model complet dans cette PR;
- pas de persistance DB ni API HTTP;
- pas de normalisation automatique silencieuse des matrices invalides.

2. Fondement mathématique obligatoire

Soit un ensemble fini d'états $S = \{s_1, s_2, \dots, s_n\}$. Une matrice de Markov P contient une probabilité de transition pour chaque couple source/cible. Chaque ligne est une distribution complète de probabilité.

$$P = [p_{ij}], \quad p_{ij} = \mathbb{P}(X_{t+1} = s_j \mid X_t = s_i)$$

Equation 1 - Définition d'une matrice de transition.

$$\forall i, \quad 0 \leq p_{ij} \leq 1, \quad \sum_{j=1}^n p_{ij} = 1$$

Equation 2 - Contraintes de validité de chaque ligne.

Le moteur doit également supporter l'évolution d'une distribution complète. Cette opération est indispensable pour modéliser une tendance interne persistante, par exemple l'état émotionnel latent d'un PNJ ou la tonalité dominante d'une run.

$$\pi_{t+1} = \pi_t P$$

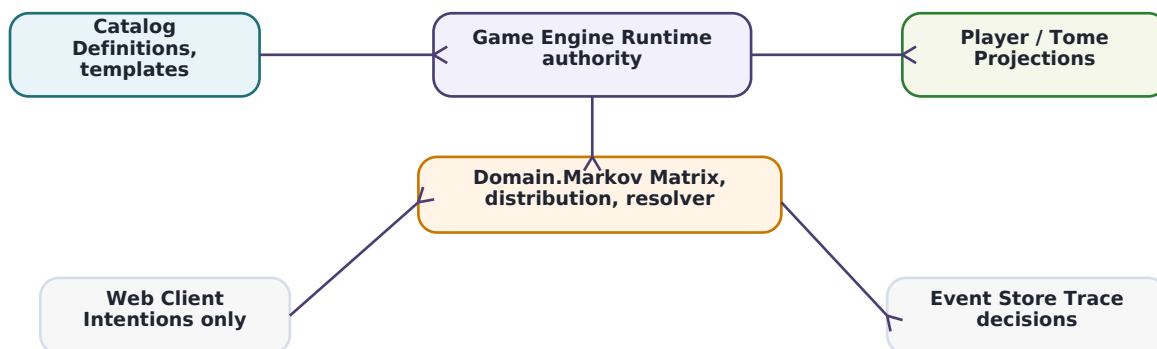
Equation 3 - Evolution d'une distribution par multiplication matricielle.

Toute implémentation qui ne sait pas calculer $\pi(t+1) = \pi_t \times P$ est insuffisante pour ce projet.

3. Positionnement architectural

Le Markov Engine appartient au Game Engine Domain. Il représente un mécanisme de décision runtime serveur-autoritaire. Il ne doit pas être placé dans shared-building-blocks, car il est trop central pour le gameplay et pas un simple utilitaire technique transversal.

Figure 1 - Positionnement du Markov Engine dans l'architecture cible



Dépendances interdites

- Leds.Catalog.*
- Entity Framework Core
- ASP.NET Core
- MediatR dans le noyau Domain
- Random.Shared, new Random(), DateTime.UtcNow, Guid.NewGuid()

Dépendances autorisées

- System.Collections.Generic
- System.Linq
- System.Security.Cryptography pour le sampler déterministe
- System.Text

4. Modèle conceptuel final cible

Le MVP doit être proche de l'implémentation finale. Les noms ci-dessous ne sont pas des placeholders : ils constituent la frontière stable attendue du domaine Markov.

Composant	Responsabilité	Statut MVP
MarkovState	Etat fini de la chaîne, identifié par une valeur stable.	Inclus
MarkovStateSet	Collection validée des états de la matrice, utile pour éviter les doublons.	Optionnel MVP / cible final
MarkovTransitionRow	Ligne $P[i,*]$ de transition depuis un état source vers tous les états cibles.	Inclus
MarkovTransitionMatrix	Matrice complète P avec key, version, states et rows.	Inclus
MarkovStateDistribution	Distribution π sur les états, validée et compatible avec une matrice.	Inclus
DeterministicMarkovSampler	Produit un sample stable dans $[0,1]$ depuis les entrées déterministes.	Inclus
MarkovTransitionResolver	Résout une transition discrète via distribution cumulée.	Inclus
MarkovDistributionProjector	Applique $\pi(t+1)=\pi P$. Peut être méthode de Matrix ou service dédié.	Inclus sous forme méthode Advance

Composant	Responsabilité	Statut MVP
MarkovTransitionTrace	Trace auditable d'une décision : sample, source, cible, version.	Prévu dès modèle, peut être PR suivante
MarkovMatrixModifier	Composition future par Lois du Palais.	Exclu MVP

Structure cible de fichier

```
services/game-engine/src/Leds.GameEngine.Domain/Markov/
  MarkovState.cs
  MarkovTransitionRow.cs
  MarkovTransitionMatrix.cs
  MarkovStateDistribution.cs
  MarkovMatrixValidationException.cs
  DeterministicMarkovSampler.cs
  MarkovTransitionResolver.cs
  MarkovTransitionTrace.cs      # cible proche finale
```

5. Etats narratifs et garde-fous non médicaux

Le Palais est une représentation poétique de la psyché, mais le jeu n'est pas un dispositif médical. Les états Markov utilisés pour les PNJ peuvent inclure des émotions humaines et des états psychologiques narratifs connus, mais ils ne doivent pas devenir des diagnostics ni des maladies psychiatriques.

Règle de nommage

Un état doit décrire une posture narrative observable ou une dynamique émotionnelle jouable. Il ne doit pas nommer une pathologie, un trouble psychiatrique, un diagnostic ou une catégorie médicale stricte.

Catégorie	Etats autorisés / recommandés	A éviter
Emotions de base	Joy, Sadness, Fear, Anger, Disgust, Surprise, Calm	Diagnostic médical ou symptômes codés comme maladie
Attitudes relationnelles	Wary, Curious, Open, Withdrawn, Protective, Defiant, Trusting	Etiquettes cliniques ou stigmatisantes
Etats psychologiques narratifs	Dissociated, Fragmented, Shocked, TraumaticEcho, Avoidant, Lucid	PTSD, schizophrenia, bipolar, depression comme états de gameplay
Etats du Palais	Silenced, Ruptured, Mirrored, Threshold, Memory, Collapse, Renewal	Terminologie médicale présentée comme vérité clinique

Catalogue initial recommandé pour les attitudes PNJ

Etat	Usage narratif	Note de sécurité narrative
Calm	Etat stable, réponses mesurées.	Emotion générale, non médicale.
Wary	Méfiance, prudence, retenue.	Attitude relationnelle.
Curious	Ouverture progressive à l'échange.	Emotion/attitude.
Open	Confiance suffisante pour révéler un fragment.	Attitude narrative.
Withdrawn	Retrait, réponse minimale.	Pas de diagnostic.
Dissociated	Distance au réel du Palais, fragmentation de la scène.	A utiliser poétiquement, sans discours médical.
Fragmented	Souvenir incomplet, parole brisée.	Etat narratif du Palais.
TraumaticEcho	Résonance d'un événement passé.	Ne pas présenter comme diagnostic.
Hostile	Rejet, menace, combat possible.	Comportement observable.
Lucid	Clarté, révélation, vérité temporaire.	Etat narratif.

6. Déterminisme et reproductibilité

Chaque décision Markov doit être reproductible. Le sampler convertit des entrées stables en un nombre dans $[0,1)$. Le sample est ensuite appliqué à la distribution cumulée de la ligne courante.

Entrée	Rôle
seed	Seed de run ou seed dérivée.
scope	Sépare les flux : palace-generation, npc-attitude, narrative-tone, himlit-adaptation.
matrixKey	Identifie la matrice logique.
matrixVersion	Garantit la stabilité des seeds après évolution du contenu.
currentState	Etat source.
step	Index déterministe de transition dans le scope.

Algorithme sampler cible

```
input = seed + "|" + scope + "|" + matrixKey + "|" + matrixVersion + "|" + currentState + "|" + step
hash = SHA256(input)
first64bits = UInt64(hash[0..8])
sample = first64bits / (UInt64.MaxValue + 1)
assert 0 <= sample < 1
```

7. Validation des matrices

La validation est stricte. Le MVP ne normalise pas automatiquement une matrice invalide. Une ligne dont la somme ne vaut pas 1 est rejetée. Une normalisation future devra être explicite, testée et traçable.

Invariant	Comportement attendu
Key vide	Rejet.
Version vide	Rejet.
Aucun état	Rejet.
Aucune ligne	Rejet.
Etat dupliqué	Rejet.
Ligne dupliquée	Rejet.
Ligne manquante	Rejet.
Cible manquante	Rejet.
Cible inconnue	Rejet.
Probabilité < 0	Rejet.
Probabilité > 1	Rejet.
Somme de ligne != 1	Rejet avec tolérance 0.000001.
Distribution incompatible	Rejet.

Tolérance

La tolérance de comparaison recommandée est 0.000001. Elle anticipe les futures compositions de matrices, tout en restant suffisamment stricte pour détecter les erreurs d'équilibrage.

8. Opérations Markov obligatoires

8.1 Transition discrète observable

Cette opération produit un état observable à partir de l'état courant. Elle sera utilisée par la génération, les événements ou les attitudes PNJ.

```
nextState = resolver.ResolveNextState(
```

```
matrix,  
currentState,  
seed,  
scope,  
step);
```

8.2 Evolution complète de distribution

Cette opération conserve l'information probabiliste complète. Elle est nécessaire pour les tendances internes, les profils de run, l'adaptation d'Him'Lit et les analyses post-run.

```
nextDistribution = matrix.Advance(currentDistribution);  
// equivalent math:  $\pi(t+1) = \pi(t) \times P$ 
```

8.3 Distribution de Dirac

Le moteur doit permettre de convertir un état discret en distribution initiale concentrée sur cet état.

```
CurrentState = Wary  
Distribution = { Calm:0, Wary:1, Curious:0, Open:0, ... }
```

9. Diagrammes de fonctionnement

Figure 2 - Pipeline de transition discrète

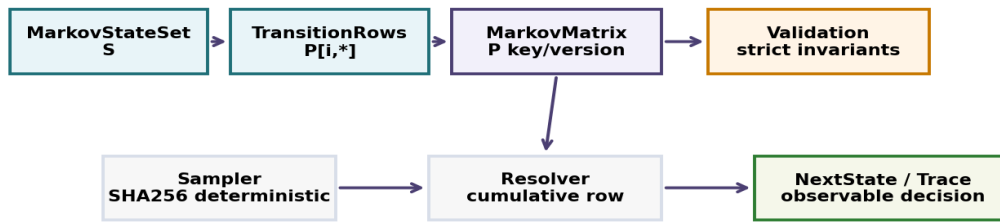


Figure 2 - Pipeline de transition discrète.

Figure 3 - Exemple de matrice P pour un profil de PNJ fragmentaire

	Calm	Wary	Curious	Open	Dissoc.	Hostile	Lucid
Calm	0.45	0.20	0.20	0.05	0.05	0.00	0.05
Wary	0.10	0.45	0.20	0.05	0.10	0.10	0.00
Curious	0.10	0.15	0.35	0.20	0.10	0.05	0.05
Open	0.10	0.05	0.20	0.35	0.05	0.00	0.25
Dissoc.	0.05	0.20	0.10	0.05	0.35	0.15	0.10
Hostile	0.00	0.20	0.05	0.00	0.20	0.45	0.10
Lucid	0.10	0.05	0.15	0.25	0.05	0.00	0.40

Chaque ligne contient tous les états cibles et somme à 1.

Figure 3 - Exemple non définitif de matrice d'attitude. Elle illustre la structure mathématique, pas un équilibrage final.

Exemple calculé : $\pi(t+1) = \pi_t P$

Matrice P	Calm	Wary
Calm	0.70	0.30
Wary	0.20	0.80

Distribution initiale	Calm	Wary
π_t	0.50	0.50
$\pi(t+1)$	$0.50 \cdot 0.70 + 0.50 \cdot 0.20 = 0.45$	$0.50 \cdot 0.30 + 0.50 \cdot 0.80 = 0.55$

10. API domaine cible

L'implémentation doit être proche de la version finale. Le code pourra évoluer, mais les responsabilités ci-dessous doivent rester stables.

```
public sealed record MarkovState(string Value);

public sealed class MarkovTransitionRow
{
    public MarkovState SourceState { get; }
    public IReadOnlyDictionary<MarkovState, decimal> Probabilities { get; }
}

public sealed class MarkovTransitionMatrix
{
    public string Key { get; }
    public string Version { get; }
    public IReadOnlyCollection<MarkovState> States { get; }
    public IReadOnlyDictionary<MarkovState, MarkovTransitionRow> Rows { get; }

    public MarkovStateDistribution Advance(MarkovStateDistribution distribution);
}

public sealed class MarkovTransitionResolver
{
    public MarkovState ResolveNextState(
        MarkovTransitionMatrix matrix,
        MarkovState currentState,
        string seed,
        string scope,
        int step);

    public MarkovState ResolveNextState(
        MarkovTransitionMatrix matrix,
        MarkovState currentState,
        decimal sample);
}
```

Trace cible

```
public sealed record MarkovTransitionTrace(
    string MatrixKey,
    string MatrixVersion,
    string Scope,
    string Seed,
    int Step,
    string SourceState,
    string TargetState,
    decimal Sample);
```

11. Traces, Event Sourcing et auditabilité

Le moteur Markov ne persiste rien directement. En revanche, ses décisions doivent pouvoir être tracées par les modules de run afin de reconstruire ou expliquer une génération.

Champ trace	Rôle
matrixKey / matrixVersion	Rejouer avec la bonne matrice.
scope	Identifier le contexte : generation, npc-attitude, narrative-tone.

Champ trace	Rôle
seed	Reproductibilité.
step	Position de la transition dans le flux.
sourceState / targetState	Audit de la transition.
sample	Explication exacte du choix dans la distribution cumulée.

12. Relation avec Catalog et Lois du Palais

12.1 Catalog

Catalog pourra stocker des définitions de matrices ou des profils qui référencent une matrixKey. Le Game Engine reste responsable de la résolution runtime. Aucune entité Domain Catalog ne doit être consommée directement par Domain.Markov.

12.2 Lois du Palais

Les Lois du Palais ne sont pas appliquées dans le MVP. Le moteur doit toutefois permettre une composition future de matrice effective. Toute matrice modifiée par une Loi devra être revalidée.

$$P_{effective} = Normalize(P_{base} \odot W_{context})$$

Equation 4 - Composition future contrôlée par contexte ou Lois du Palais.

Règle future

Une Loi ne doit jamais produire une matrice effective invalide. Après modification, chaque ligne doit toujours sommer à 1.

13. Stratégie de tests

Famille de tests	Cas obligatoires
Validation	matrice vide, état vide, version vide, ligne manquante, ligne dupliquée, cible inconnue, cible manquante, probabilité hors bornes, somme de ligne invalide
Distribution	distribution vide, somme invalide, état manquant, état inconnu, distribution de Dirac
Transition discrète	sample dans premier intervalle, intervalle intermédiaire, sample proche de 0, sample proche de 1, sample = 1 rejeté
Déterminisme	même entrée = même sample, changement de step/scope/version = sample potentiellement différent
Multiplication matricielle	exemple calculé manuellement pour $\pi(t+1)=\pi tP$
Architecture	aucune référence Catalog, aucune dépendance infrastructure, aucun Random.Shared

14. Roadmap d'intégration

Ordre	Jalon	Objectif	Commit possible
1	Noyau Markov	Matrice, validation, distribution, déterminisme.	feat(game-engine): introduce deterministic markov engine
2	Génération du Palais	Guider thèmes de room avec P.	feat(game-engine): connect room generation to markov matrix
3	Types d'événements	Sélection Markov puis contraintes serveur.	feat(game-engine): select node event types with markov matrix
4	Attitudes PNJ	Etats émotionnels et psychologiques narratifs.	feat(game-engine): add markov npc attitude profiles
5	Lois du Palais	Modifier P en matrice effective revalidée.	feat(game-engine): apply palace law modifiers to markov matrices
6	Him'Lit	Déduire adaptation depuis distributions de run.	feat(game-engine): derive himlit adaptation from markov traces

15. Critères d'acceptation

- Domain/Markov contient le noyau mathématique sans logique PNJ, Room, Node ou Catalog.
- MarkovTransitionMatrix représente une vraie matrice P avec key et version.
- Chaque ligne contient tous les états cibles et somme à 1.
- MarkovStateDistribution supporte π et l'opération $\pi(t+1)=\pi tP$.
- MarkovTransitionResolver choisit un état par distribution cumulée.
- DeterministicMarkovSampler ne dépend d'aucune source aléatoire ou temporelle non contrôlée.
- Le même seed et les mêmes entrées produisent le même résultat.
- Les tests prouvent les invariants mathématiques et le déterminisme.
- Aucune pathologie psychiatrique ou diagnostic médical n'est encodé comme état gameplay.
- Les états psychologiques sont utilisés comme états narratifs du Palais, pas comme vérité clinique.

Décision finale

Le Markov Engine est accepté uniquement s'il permet au minimum les deux opérations suivantes : transition discrète déterministe depuis un état courant, et évolution complète d'une distribution par multiplication matricielle. Toute implémentation inférieure doit être refusée.

Références projet

- SFD v2.0.0 - cadrage de la génération par seed, generatorVersion et markovMatrixVersion.
- SFD v2.0.0 - règles serveur-autoritaire : le frontend transmet des intentions, le backend décide.
- Spécification de transition V1 vers V2 - Game Engine comme runtime central et Catalog comme référentiel de contenu versionné.
- Directive projet - la matrice doit être une vraie implémentation mathématique, testable, déterministe et versionnable.